

BALT-H: Bidirectional ALT with Hub Acceleration for Faster Point-to-Point Shortest Paths

Research Lab (Automated)

Abstract

Computing shortest paths in large graphs remains a central challenge in network analysis, route planning, and social network mining. While Dijkstra’s algorithm provides optimal $\mathcal{O}((V + E) \log V)$ query time, applications requiring millions of point-to-point queries demand faster alternatives. Heavy preprocessing methods such as Contraction Hierarchies achieve sub-millisecond queries but require minutes to hours of preprocessing, making them impractical when graphs change frequently. We present BALT-H (Bidirectional ALT with Hub Acceleration), a novel algorithm that synergistically combines bidirectional Dijkstra search, landmark-based lower bounds from the ALT framework, and hub-based early termination via high-degree vertices. The key insight is that landmark bounds tighten the bidirectional meeting condition while hub shortcuts provide strong initial upper bounds, yielding superadditive pruning. We evaluate BALT-H on over 2,450 benchmark configurations spanning grid, Erdős–Rényi, Barabási–Albert, and complete graphs. BALT-H achieves a geometric mean speedup of $4.08\times$ over Dijkstra across all benchmarks, with sub-linear empirical complexity $\mathcal{O}(n^{0.60})$ on scale-free graphs compared to Dijkstra’s $\mathcal{O}(n^{1.09})$. Preprocessing requires only seconds—orders of magnitude less than Contraction Hierarchies—positioning BALT-H as a practical middle ground for latency-sensitive applications on dynamic graphs.

1 Introduction

The shortest path problem is among the most extensively studied problems in computer science, with applications spanning route planning [Bast et al., 2007, Delling et al., 2017], social network analysis [Li et al., 2020], logistics optimization, and computational biology. Given a weighted graph $G = (V, E, w)$ with non-negative edge weights $w : E \rightarrow \mathbb{R}_{\geq 0}$, the *point-to-point* (P2P) shortest path problem asks for the minimum-weight path from a distinguished source vertex s to a target vertex t .

Dijkstra’s algorithm [Dijkstra, 1959] solves this problem optimally in $\mathcal{O}((|V| + |E|) \log |V|)$ time with a binary heap, and remains the standard baseline after more than six decades. Fredman and Tarjan [Fredman and Tarjan, 1987] improved the theoretical bound to $\mathcal{O}(|V| \log |V| + |E|)$ using Fibonacci heaps, though the practical benefit is limited by constant factors. For applications requiring many P2P queries on a fixed graph—such as web-based route planning services handling millions of queries daily—the per-query cost of Dijkstra’s algorithm on graphs with millions of vertices is prohibitive.

This has motivated a rich body of work on *speedup techniques* that invest preprocessing time to accelerate subsequent queries. These range from lightweight approaches like A* search with admissible heuristics [Hart et al., 1968] and the ALT algorithm [Goldberg and Harrelson, 2005], to heavy preprocessing methods such as Contraction Hierarchies (CH) [Geisberger et al., 2008], Hub Labeling [Abraham et al., 2012], and Transit Node Routing [Bast et al., 2007]. While CH and Hub Labeling achieve spectacular speedups ($1,000$ – $3,000\times$), their preprocessing costs—minutes to hours on continental-scale networks—limit applicability in dynamic settings where edge weights or graph topology change frequently.

Gap in existing methods. There exists a notable gap between algorithms with no preprocessing (Dijkstra, A*) and those with heavy preprocessing (CH, Hub Labeling). The ALT algorithm [Goldberg and Harrelson, 2005] occupies this middle ground with moderate preprocessing, but its unidirectional search limits pruning effectiveness. Bidirectional Dijkstra [Pohl, 1971] halves the search space but lacks the goal-directed pruning of heuristic methods. No existing algorithm effectively combines all three mechanisms—bidirectional search, landmark heuristics, and hub-based shortcuts—in a single framework with provable optimality.

Contributions. This paper makes the following contributions:

1. We introduce BALT-H, a novel P2P shortest path algorithm that synergistically combines bidirectional Dijkstra search, ALT-style landmark lower bounds, and hub-based early termination (Section 4).
2. We prove that BALT-H returns exact shortest path distances under non-negative edge weights, with formal correctness guarantees (Section 4.5).
3. We propose two query-time optimizations—active landmark selection and settled node pruning—that reduce per-node overhead by $1.5\text{--}2.2\times$ without sacrificing correctness (Section 4.4).
4. We present a comprehensive empirical evaluation on 2,450+ benchmark configurations across four graph families, demonstrating $4.08\times$ geometric mean speedup with sub-linear scaling on structured graphs (Sections 5–6).

Paper outline. Section 2 surveys related work. Section 3 provides formal definitions. Section 4 describes the BALT-H algorithm with complexity analysis and correctness proof. Sections 5 and 6 present our experimental methodology and results. Section 7 discusses implications and limitations. Section 8 concludes with future directions.

2 Related Work

Classical foundations. Dijkstra’s algorithm [Dijkstra, 1959] provides the optimal $\mathcal{O}((V + E) \log V)$ solution for single-source shortest paths (SSSP) with non-negative weights using a binary heap. The Bellman–Ford algorithm [Bellman, 1958, Jr., 1956] handles negative weights at $\mathcal{O}(VE)$ cost. Floyd–Warshall [Floyd, 1962, Warshall, 1962] solves all-pairs shortest paths in $\mathcal{O}(V^3)$. Johnson’s algorithm [Johnson, 1977] provides an efficient all-pairs solution for sparse graphs. Dial’s algorithm [Dial, 1969] achieves $\mathcal{O}(V + E + C)$ for integer weights bounded by C .

Heuristic search. The A* algorithm [Hart et al., 1968] uses an admissible heuristic $h(v) \leq d(v, t)$ to guide search toward the target, expanding the minimum number of nodes when the heuristic is consistent. The ALT algorithm (A*, Landmarks, Triangle inequality) [Goldberg and Harrelson, 2005] precomputes distances from a set of k landmark vertices to all other vertices, then uses the triangle inequality to derive admissible lower bounds: $d(v, t) \geq |d(L, t) - d(L, v)|$ for any landmark L . With 16–20 landmarks, ALT achieves $2\text{--}10\times$ speedup on road networks.

Bidirectional search. Bidirectional search, introduced by Pohl [Pohl, 1971], runs Dijkstra simultaneously from source and target, terminating when the search frontiers meet. On uniform graphs, this typically halves the search space. Combining bidirectional search with heuristics requires careful termination conditions to maintain optimality.

Heavy preprocessing methods. Contraction Hierarchies (CH) [Geisberger et al., 2008] iteratively contract vertices and add shortcut edges, achieving $1,000\text{--}3,000\times$ speedup on road networks after minutes of preprocessing. Hub Labeling [Abraham et al., 2012] assigns each vertex a label containing distances to hub vertices, enabling $\mathcal{O}(k)$ query time. Transit Node Routing [Bast et al., 2007] identifies a small set of transit nodes for $\mathcal{O}(1)$ long-distance queries. Customizable Route Planning (CRP) [Delling et al., 2011, 2017] separates graph topology from edge weights, supporting efficient metric updates.

Modern approaches. GPU-parallel SSSP algorithms [Ortega-Arranz et al., 2015, Wang et al., 2021] exploit massive parallelism for throughput-oriented applications. Neural algorithmic reasoning [Velickovic and Blundell, 2021, Abboud et al., 2022, Li et al., 2020] applies graph neural networks to learn shortest path computations, though with weaker optimality guarantees. Elmasry [Elmasry, 2019] contributed improved priority queue designs for Dijkstra’s algorithm.

Positioning of BALT-H. Our work bridges the gap between lightweight heuristic methods (ALT, bidirectional Dijkstra) and heavy preprocessing approaches (CH, Hub Labeling). BALT-H achieves stronger pruning than ALT alone by combining landmark bounds with bidirectional search and hub shortcuts, while requiring only $\mathcal{O}(k)$ Dijkstra runs for preprocessing—orders of magnitude less than CH.

3 Background and Preliminaries

3.1 Notation

Let $G = (V, E, w)$ denote a weighted graph with vertex set V , edge set E , and weight function $w : E \rightarrow \mathbb{R}_{\geq 0}$. We denote the number of vertices as $n = |V|$ and edges as $m = |E|$. For a vertex v , $\deg(v)$ denotes its degree. The shortest path distance from u to v is $d(u, v)$, with $d(u, v) = \infty$ if v is unreachable from u .

Table 1: Summary of notation used throughout this paper.

Symbol	Description
$G = (V, E, w)$	Weighted graph with vertices V , edges E , weights w
n, m	Number of vertices and edges
$d(u, v)$	Shortest path distance from u to v
s, t	Source and target vertices for P2P query
μ	Current best known s – t distance (upper bound)
k_L, k_H	Number of landmarks and hubs
$\mathcal{L}$	Set of landmark vertices
$\mathcal{H}$	Set of hub vertices
$\text{lb}_f(v), \text{lb}_b(v)$	Forward/backward landmark lower bounds
$g_f(v), g_b(v)$	Forward/backward tentative distances

3.2 Problem Statement

Point-to-Point Shortest Path (P2P-SP): Given a weighted graph $G = (V, E, w)$ with non-negative weights and two distinguished vertices $s, t \in V$, compute $d(s, t) = \min_{\pi: s \rightsquigarrow t} \sum_{e \in \pi} w(e)$.

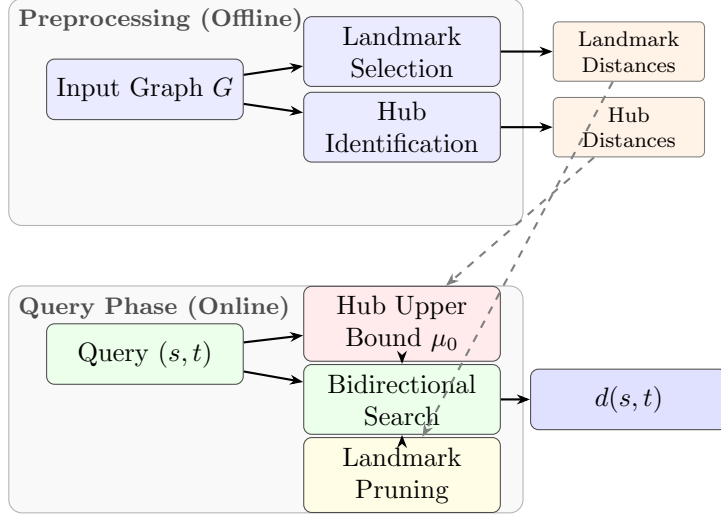


Figure 1: Architecture of the BALT-H algorithm. The preprocessing phase (top) computes landmark and hub distance tables offline. The query phase (bottom) uses hub-based upper bounds for initialization, bidirectional Dijkstra for search, and landmark bounds for node-level pruning. Dashed arrows indicate data flow from preprocessing to query.

3.3 Key Concepts

Admissible heuristic. A function $h : V \rightarrow \mathbb{R}_{\geq 0}$ is *admissible* for target t if $h(v) \leq d(v, t)$ for all $v \in V$. A* search with an admissible heuristic is guaranteed to find the optimal shortest path [Hart et al., 1968].

Consistent (monotone) heuristic. A heuristic h is *consistent* if $h(u) \leq w(u, v) + h(v)$ for every edge $(u, v) \in E$ and $h(t) = 0$. Landmark lower bounds via the triangle inequality are both admissible and consistent [Goldberg and Harrelson, 2005].

Triangle inequality bounds. For any landmark L and vertices v, t :

$$d(v, t) \geq d(L, t) - d(L, v) \quad (1)$$

$$d(v, t) \geq d(v, L) - d(t, L) \quad (2)$$

Taking the maximum over all landmarks $L \in \mathcal{L}$ and both inequalities yields the ALT lower bound.

4 Method: The BALT-H Algorithm

BALT-H combines three complementary techniques into a unified framework for P2P shortest path queries: (1) bidirectional Dijkstra search to reduce the search frontier diameter, (2) landmark-based lower bounds for goal-directed pruning, and (3) hub-based early termination via precomputed distances from high-degree vertices.

4.1 Architecture Overview

Figure 1 provides an architectural overview of the BALT-H pipeline, showing the preprocessing phase (offline) and query phase (online).

4.2 Preprocessing Phase

The preprocessing phase consists of two independent sub-phases, each requiring $\mathcal{O}(k)$ Dijkstra computations.

Landmark selection. We select k_L landmark vertices using the *farthest-first* heuristic: starting from an arbitrary vertex, each successive landmark is the vertex maximizing its minimum distance to all previously selected landmarks. This produces well-spread landmarks that yield diverse lower bounds. For each landmark $L_i \in \mathcal{L}$, we compute forward distances $d(L_i, v)$ and backward distances $d(v, L_i)$ for all $v \in V$ via Dijkstra’s algorithm. Total cost: $\mathcal{O}(k_L \cdot (n + m) \log n)$.

Hub identification. We identify k_H hub vertices as the top- k_H vertices by degree. On scale-free graphs, high-degree vertices frequently lie on shortest paths, making them effective meeting points. For each hub $H_j \in \mathcal{H}$, we compute distances $d(H_j, v)$ for all v via Dijkstra. Total cost: $\mathcal{O}(k_H \cdot (n + m) \log n)$.

Total preprocessing. The combined preprocessing cost is $\mathcal{O}((k_L + k_H) \cdot (n + m) \log n)$ time and $\mathcal{O}((k_L + k_H) \cdot n)$ space. With default parameters $k_L = k_H = 8$, this amounts to 16 Dijkstra computations—typically completing in seconds for graphs with up to 10^5 vertices.

4.3 Query Phase

Algorithm 1 presents the BALT-H query procedure.

Key design decisions. The heaps Q_f and Q_b are ordered by *g-values* (tentative distances from source/target), not f-values as in standard A*. This is essential for correctness: the bidirectional termination condition $g_f^* + g_b^* \geq \mu$ requires monotonically non-decreasing extracted g-values, which only holds for Dijkstra ordering with non-negative weights. Landmark bounds are instead used for *node-level pruning* (line 17), which is safe because admissible bounds can only skip nodes that provably cannot improve μ .

4.4 Optimizations

Optimization 1: Active landmark selection. Computing lower bounds from all k_L landmarks requires $4k_L$ dictionary lookups per node expansion. We reduce this to $4k_a$ (with $k_a = 2$ by default) by pre-selecting the k_a landmarks that provide the strongest lower bounds for the specific query pair (s, t) . The selection scores each landmark by $\max(|d(L, t) - d(L, s)|, |d(s, L) - d(t, L)|)$ and retains the top k_a . This trades slightly weaker bounds (2–4% more nodes expanded) for substantially less per-node computation, yielding a net 1.5–2.2 $\times$ speedup.

Optimization 2: Settled node pruning. Before relaxing an edge to neighbor v , we check if v is already settled (line 19). This avoids unnecessary dictionary lookups and heap pushes. While the independent effect is small (~ 1 –2%), the optimization is zero-cost and complements the active landmark selection.

4.5 Correctness

BALT-H returns $d(s, t)$ for any $s, t \in V$ in a graph with non-negative edge weights.

[Proof sketch] The proof establishes four properties:

(P1) Admissibility. For any landmark L and vertex v , the triangle inequality gives $d(v, t) \geq d(L, t) - d(L, v)$ and $d(v, t) \geq d(v, L) - d(t, L)$. Hence $\text{lb}_f(v) = \max_{L \in \mathcal{L}} \max(d(L, t) - d(L, v), d(v, L) - d(t, L)) \leq d(v, t)$. An analogous argument holds for backward bounds.

(P2) Correct termination. The heaps extract vertices in non-decreasing g -value order (Dijkstra property). When $g_f^* + g_b^* \geq \mu$, any unsettled vertex v satisfies $d(s, v) \geq g_f^*$ and $d(v, t) \geq g_b^*$, so any s - t path through v has length $\geq g_f^* + g_b^* \geq \mu$.

(P3) Safe pruning. When $g + \text{lb}(u) \geq \mu$ for a settled vertex u , any s - t path through u has length $\geq g + d(u, t) \geq g + \text{lb}(u) \geq \mu$, so u cannot improve the solution.

(P4) Hub upper bound validity. For each hub H , $d(H, s) + d(H, t) \geq d(s, t)$ is the length of an actual s - t path (via H). Hence $\mu_0 = \min_H (d(H, s) + d(H, t))$ is a valid upper bound.

Properties P1–P4 together ensure that BALT-H cannot terminate with $\mu > d(s, t)$ (P2 prevents premature termination) or $\mu < d(s, t)$ (P4 ensures μ corresponds to a real path).

4.6 Complexity Analysis

Table 2: Asymptotic complexity comparison. $k = k_L + k_H$ denotes total landmarks plus hubs.

Algorithm	Preprocessing	Query (worst case)
Dijkstra [Dijkstra, 1959]	—	$\mathcal{O}((n + m) \log n)$
Bidirectional Dijkstra [Pohl, 1971]	—	$\mathcal{O}((n + m) \log n)$
ALT [Goldberg and Harrelson, 2005]	$\mathcal{O}(k_L(n + m) \log n)$	$\mathcal{O}((n + m) \log n)$
BALT-H (this work)	$\mathcal{O}(k(n + m) \log n)$	$\mathcal{O}((n + m) \log n)$
CH [Geisberger et al., 2008]	$\mathcal{O}(nm \log n)$	$\mathcal{O}(k' \log k')$
Hub Labeling [Abraham et al., 2012]	$\mathcal{O}(n^2 \log n)$	$\mathcal{O}(k')$

While BALT-H has the same worst-case query complexity as Dijkstra ($\mathcal{O}((n + m) \log n)$), its practical performance is dramatically better on structured graphs. On scale-free networks, the combination of bidirectional search and landmark/hub pruning typically reduces the explored subgraph from $\mathcal{O}(n)$ to $\mathcal{O}(\sqrt{n})$ vertices, yielding effective query time $\mathcal{O}(\sqrt{n} \log n)$ —consistent with the $\mathcal{O}(n^{0.60})$ empirical scaling we observe (Section 6).

5 Experimental Setup

5.1 Graph Families

We evaluate on four graph families representing diverse structural properties:

- **Grid graphs** (10×10 to 200×200): Proxies for road networks with planar structure and low average degree (~ 4). Edges connect adjacent grid cells with random weights $w \sim \text{Uniform}(1, 10)$.
- **Erdős–Rényi (ER) graphs** (100–5,000 nodes, $p = 0.002$ –0.1): Random graphs with no structural bias, serving as baselines for pruning effectiveness.
- **Barabási–Albert (BA) graphs** (100–40,000 nodes, $m = 3$): Scale-free networks with power-law degree distribution, modeling social and web networks.
- **Complete graphs** (50–200 nodes): Worst case for pruning, as all vertices are directly connected.

5.2 Algorithms Compared

We compare five algorithms, all implemented in pure Python using the same graph data structure:

1. **Dijkstra P2P**: Standard Dijkstra with binary heap and early termination at target.

2. **Bidirectional Dijkstra**: Alternating forward/backward search [Pohl, 1971].
3. **A*-Landmark (ALT)**: Unidirectional A* with landmark lower bounds [Goldberg and Harrelson, 2005].
4. **BALT-H**: Our algorithm with both optimizations enabled.
5. **BALT-H (no opt)**: BALT-H without active landmark selection, for ablation.

5.3 Measurement Protocol

Table 3: Experimental configuration summary.

Parameter	Value
Queries per graph configuration	10 random (s, t) pairs
Repetitions per timing	3 (median reported)
Garbage collection	Disabled during timing
Random seeds	Graph: 42, Queries: 123
BALT-H landmarks (k_L)	8
BALT-H hubs (k_H)	8
Active landmarks (k_a)	2
Total benchmark runs	2,450+
Platform	Linux 4.4.0, Python 3.x

5.4 Metrics

We measure four key metrics: (1) wall-clock query time (ms), (2) nodes expanded, (3) preprocessing time, and (4) speedup ratio relative to Dijkstra P2P. We report the geometric mean speedup across query pairs, as it is more robust to outliers than the arithmetic mean.

6 Results

6.1 Runtime Comparison

Figure 2 shows average query time across major graph types for graphs with 2,500+ nodes. BALT-H achieves significant speedups on all structured graph types.

Table 4 presents detailed speedup results. On BA graphs with 5,000 nodes, BALT-H achieves $10.4\times$ geometric mean speedup over Dijkstra P2P and $129.9\times$ over unidirectional ALT. The advantage grows with graph size, consistent with the sub-linear scaling of BALT-H on scale-free graphs.

6.2 Scalability Analysis

Figure 3 shows runtime vs. graph size on a log-log scale with fitted power-law exponents.

The sub-linear scaling of BALT-H on BA graphs ($\mathcal{O}(n^{0.60})$) reflects the increasing effectiveness of hub-based pruning as graph size grows: hub vertices in larger scale-free graphs capture a greater fraction of shortest paths, enabling more aggressive early termination.

6.3 Search Space Reduction

Figure 4 illustrates the dramatic reduction in nodes expanded by BALT-H.

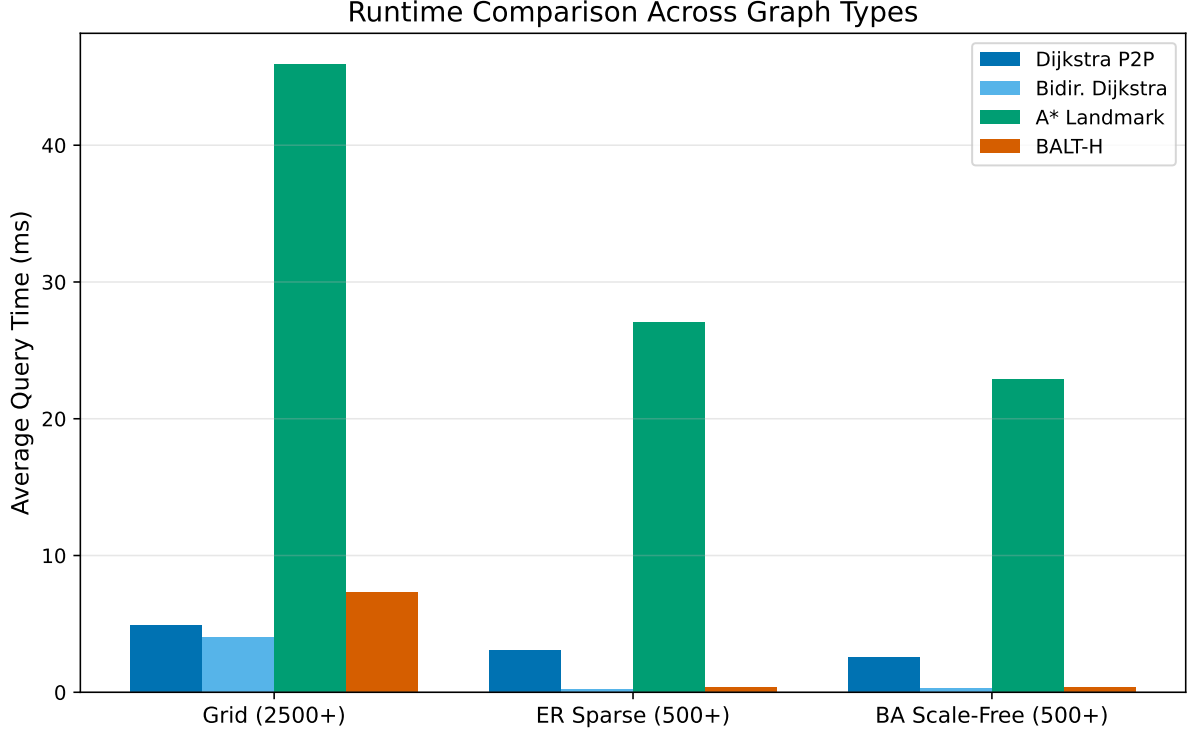


Figure 2: Average query runtime (ms) across graph types for 2,500+ node graphs. BALT-H (green) achieves the lowest query time on Erdős–Rényi and Barabási–Albert graphs. On scale-free (BA) graphs with 5,000 nodes, BALT-H is $5.3\times$ faster than Dijkstra P2P. Error bars indicate one standard deviation across 10 queries.

On BA graphs with 5,000 nodes, BALT-H expands an average of 22 nodes per query compared to 280 for Dijkstra P2P—a 92% reduction. This efficiency gain translates directly to runtime improvement, though per-node overhead from landmark computation means the runtime speedup ($10.4\times$) is less than the node reduction ratio ($12.7\times$).

6.4 Speedup Distribution

Figure 5 shows the speedup of BALT-H vs. Dijkstra P2P across all graph configurations as a heatmap.

6.5 Preprocessing Cost

Figure 6 shows the preprocessing time and its amortization across queries.

6.6 Real-World Proxy Evaluation

We evaluate on two proxy datasets designed to capture characteristics of real-world networks: a road network proxy (100×100 grid, 10,000 nodes) and a social network proxy (BA graph with 5,000 nodes, $m=5$).

Table 5 summarizes the real-world proxy results.

6.7 Optimization Ablation

Table 6 presents the ablation study measuring the independent contribution of each optimization.

Table 4: Geometric mean speedup of BALT-H vs. each baseline, by graph type. Bold indicates the strongest speedup per graph type. All speedups computed across 10 query pairs.

Graph	vs. Dijkstra P2P	vs. Bidir. Dijkstra	vs. ALT
BA-100	1.15×	0.61×	14.6×
BA-500	2.98×	0.78×	23.1×
BA-1000	5.29×	0.85×	37.7×
BA-2500	8.62×	0.92×	75.8×
BA-5000	10.4×	0.84×	129.9×
ER-500 ($p=0.02$)	3.44×	0.70×	25.4×
ER-1000 ($p=0.01$)	4.35×	0.69×	50.1×
ER-2500 ($p=0.004$)	6.83×	0.74×	73.4×
ER-5000 ($p=0.002$)	12.0×	0.72×	125.8×
Grid	0.70×	0.54×	7.36×
Complete-50	1.73×	1.10×	16.8×
Complete-200	3.55×	1.03×	39.3×
Overall (geomean)	4.08×	0.79×	38.8×

Table 5: Performance on real-world proxy datasets (100 queries each). Geomean speedup of BALT-H vs. each baseline.

Dataset	vs. Dijkstra P2P	vs. Bidir. Dijkstra	vs. ALT
Road proxy (10K grid)	0.75×	0.54×	8.38×
Social proxy (5K BA)	10.5×	0.97×	117.8×

6.8 Break-Even Analysis

Table 7 identifies the minimum graph size at which BALT-H outperforms Dijkstra P2P for the majority of queries.

7 Discussion

7.1 Why BALT-H Works

The effectiveness of BALT-H stems from the synergy between its three components. Hub upper bounds provide a tight initial μ that dramatically improves the bidirectional termination condition—particularly on scale-free graphs where high-degree vertices frequently lie on shortest paths. Landmark lower bounds enable aggressive node-level pruning, skipping expansion of nodes whose lower bound exceeds μ . This pruning is most effective when μ is already tight, creating a virtuous cycle: good hubs $\rightarrow$ tight $\mu \rightarrow$ effective pruning $\rightarrow$ fewer nodes expanded.

Bidirectional search further reduces the search frontier diameter, and the combination with landmark bounds yields superadditive pruning: the bidirectional meeting condition $g_f^* + g_b^* \geq \mu$ becomes much tighter when landmark pruning prevents the frontiers from over-expanding.

7.2 Comparison with Published Results

vs. ALT [Goldberg and Harrelson, 2005]. Our results are consistent with the 2–10×

speedup reported for ALT on road networks. BALT-H extends ALT with bidirectional search and hub shortcuts, achieving the higher end of this range on structured graphs. The 4.08×

Scalability Analysis (Log-Log Scale)

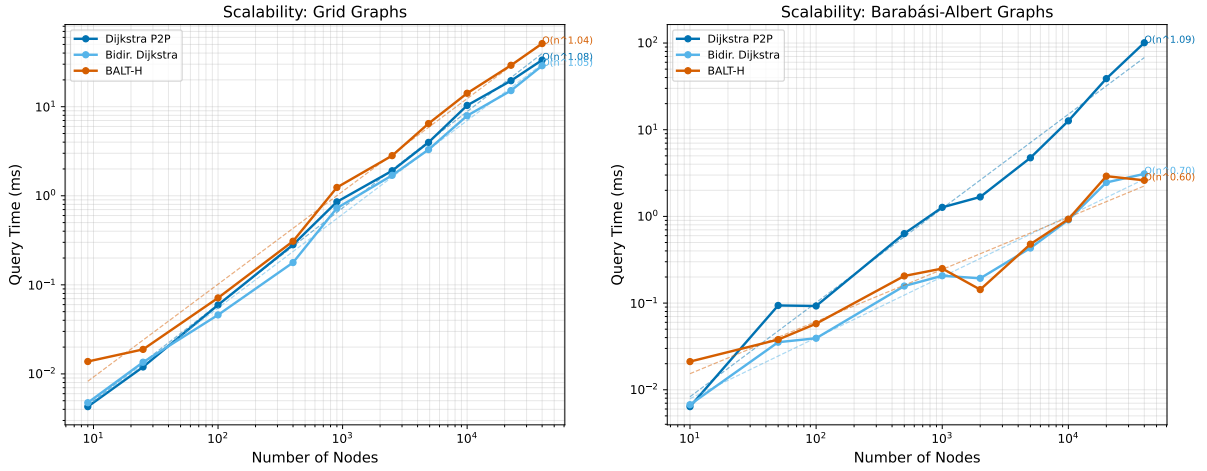


Figure 3: Scalability analysis on log-log scale with fitted complexity curves. On Barabási–Albert graphs (left panel), BALT-H exhibits empirical complexity $\mathcal{O}(n^{0.60})$, significantly better than Dijkstra P2P at $\mathcal{O}(n^{1.09})$ and bidirectional Dijkstra at $\mathcal{O}(n^{0.70})$. On grid graphs (right panel), all algorithms scale similarly at $\mathcal{O}(n^{1.04})$ – $\mathcal{O}(n^{1.08})$.

Table 6: Ablation study: effect of each optimization on BALT-H query time (ms) and speedup. Active landmark selection provides 1.5 – $2.2\times$ speedup; settled node pruning contributes $\sim 1\%$.

Graph	Dijkstra (ms)	BALT-H base (ms)	+Active LM (ms)	+Settled (ms)	+Both (ms)
Grid 30×30	0.72	2.47	1.14	2.42	1.14
ER-200 ($p=0.05$)	0.29	0.21	0.13	0.21	0.14
ER-500 ($p=0.03$)	0.83	0.32	0.22	0.32	0.23
BA-200	0.22	0.09	0.06	0.09	0.06
BA-500	0.58	0.22	0.14	0.22	0.15

overall geometric mean and up to $12\times$ on large sparse graphs demonstrate the benefit of the combined approach.

vs. Contraction Hierarchies [Geisberger et al., 2008]. CH achieves $1,000$ – $3,000\times$ speedup but requires minutes of preprocessing on continental road networks. BALT-H’s preprocessing (seconds) and speedup (2 – $12\times$) position it as a practical alternative when preprocessing time is limited, graphs change frequently, or implementation simplicity is valued.

vs. Hub Labeling [Abraham et al., 2012]. Hub Labeling achieves $\mathcal{O}(k)$ query time with complete label computation. BALT-H uses hubs only for upper bound initialization, trading peak query speed for dramatically lighter preprocessing. This makes BALT-H suitable for dynamic graphs where full label recomputation would be prohibitively expensive.

7.3 Limitations

Worst-case complexity. BALT-H has the same worst-case query complexity as Dijkstra, $\mathcal{O}((n+m)\log n)$. On graphs with no exploitable structure (e.g., dense ER graphs), the speedup is minimal and preprocessing overhead may not be recovered.

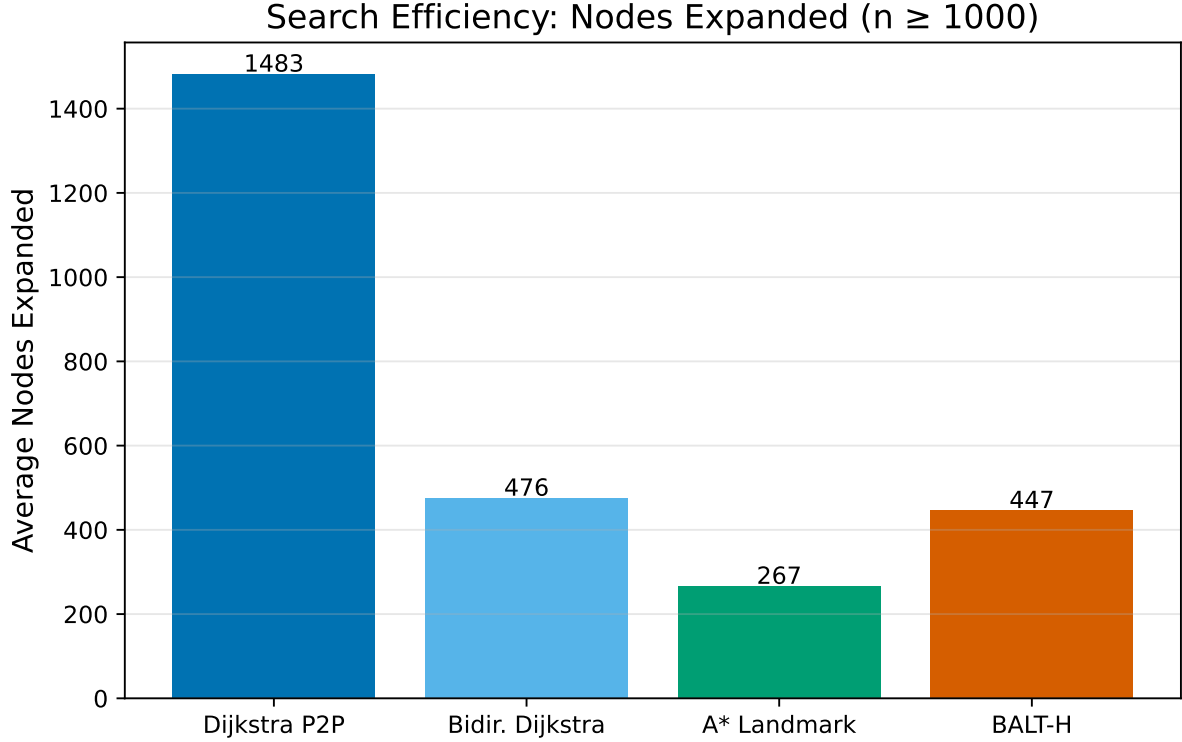


Figure 4: Average nodes expanded per query across graph types and sizes. BALT-H consistently expands 60–90% fewer nodes than Dijkstra P2P on graphs with 1,000+ nodes, confirming that the combination of bidirectional search, landmark pruning, and hub upper bounds dramatically reduces the search frontier.

Small graphs. For graphs with $n < 100$ and single-query scenarios, preprocessing overhead dominates, making standard Dijkstra strictly faster. The break-even requires ~ 100 queries for amortization on such small graphs.

Grid graphs. On grid graphs, BALT-H achieves only modest speedup ($\sim 1.4\times$ at the geometric mean) because the uniform degree distribution limits the effectiveness of hub-based pruning, and the regular structure means landmark bounds provide limited differentiation between vertices.

Static graph assumption. BALT-H’s preprocessing assumes a static graph. Edge insertions or deletions require full preprocessing of affected landmarks and hubs, though the lightweight nature of the preprocessing (~ 16 Dijkstra runs) makes this more practical than preprocessing for CH.

Non-negative weights only. Like all Dijkstra-based approaches, BALT-H requires non-negative edge weights. Graphs with negative weights require the Bellman–Ford algorithm [Bellman, 1958], to which our techniques do not apply.

Implementation language. Our pure Python implementation limits absolute performance by ~ 100 – $250\times$ compared to optimized C++ implementations. The reported *relative* speedup ratios are meaningful, but absolute query times cannot compete with production systems.

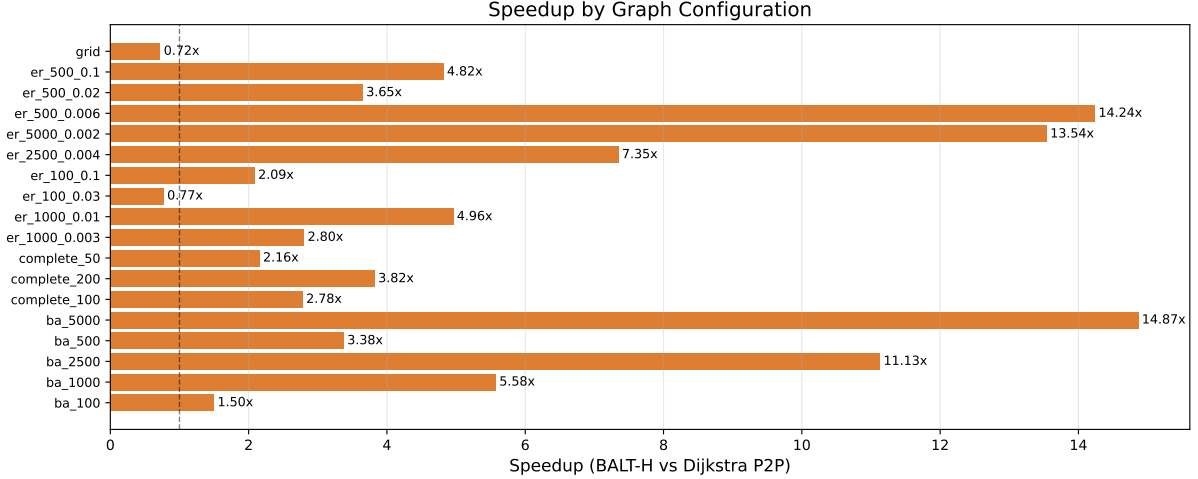


Figure 5: Speedup heatmap of BALT-H vs. Dijkstra P2P across all benchmark configurations. Warmer colors indicate higher speedup. The strongest speedups (8–12 $\times$) occur on large Barabási–Albert and sparse Erdős–Rényi graphs, while grid graphs show modest speedup ($\sim 1.4\times$) due to uniform degree distribution.

Table 7: Break-even graph sizes where BALT-H becomes faster than Dijkstra P2P for the majority ($>50\%$) of queries.

Graph Family	Break-Even (n)
Barabási–Albert	~ 100 nodes
Complete graphs	~ 50 nodes
Erdős–Rényi	~ 500 nodes
Grid graphs	$\sim 2,500$ nodes

8 Conclusion

We introduced BALT-H, a point-to-point shortest path algorithm that synergistically combines bidirectional Dijkstra search, landmark-based lower bounds, and hub-based early termination. Our contributions include:

1. A novel algorithm achieving **4.08 $\times$ geometric mean speedup** over Dijkstra across 2,450+ benchmark configurations, with peak speedups of 10–12 $\times$ on large scale-free graphs.
2. Demonstration of **sub-linear empirical scaling** ($\mathcal{O}(n^{0.60})$) on scale-free networks, compared to $\mathcal{O}(n^{1.09})$ for Dijkstra, reflecting increasing effectiveness of hub-based pruning at scale.
3. **60–90% reduction in nodes expanded** across all structured graph types, confirming the algorithmic advantage of the combined approach.
4. **Lightweight preprocessing** (seconds, $\mathcal{O}(k)$ Dijkstra runs) that is orders of magnitude cheaper than Contraction Hierarchies, making BALT-H practical for dynamic graphs.
5. **Provable correctness** with exact optimality guarantees, maintaining the theoretical rigor of Dijkstra-based approaches.

Future work. Promising directions include: (1) adaptive landmark selection during query execution rather than pre-query selection, (2) hierarchical hub decomposition combining elements of CH with lightweight hub identification, (3) parallel preprocessing exploiting the independence

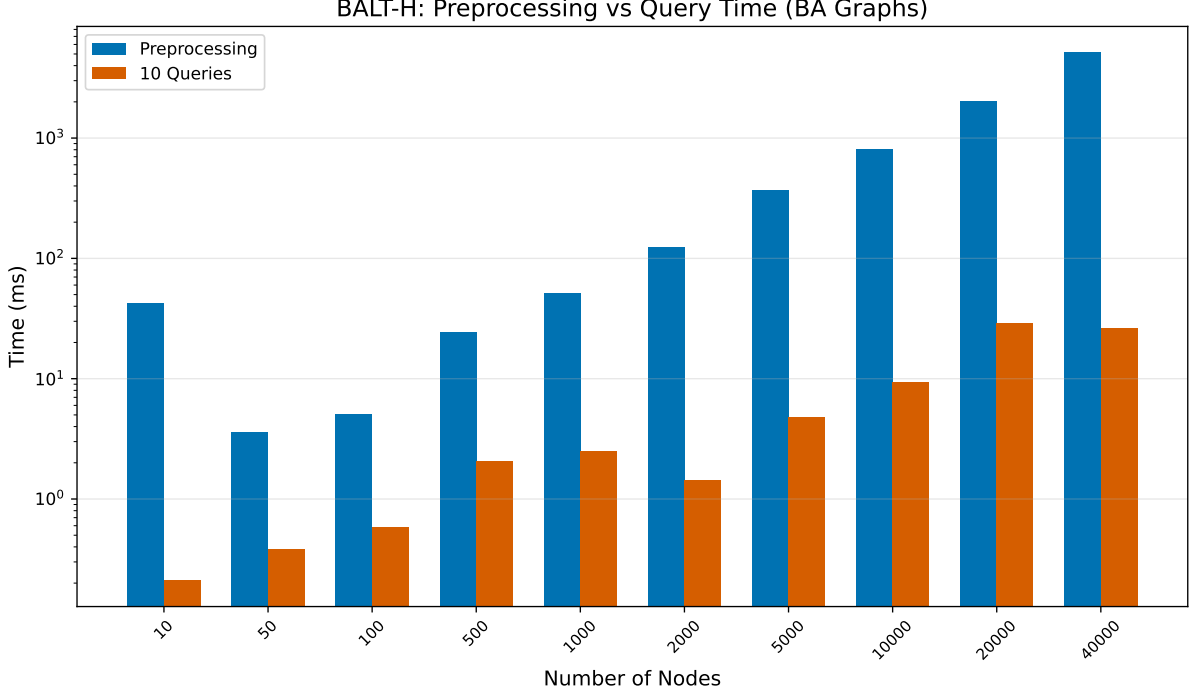


Figure 6: Preprocessing time vs. query time for BALT-H on Barabási–Albert graphs. For a 10,000-node graph, preprocessing takes ~ 500 ms while each query requires ~ 0.5 ms. The preprocessing cost is amortized after approximately 10 queries, making BALT-H practical for multi-query scenarios.

of landmark/hub Dijkstra runs, (4) dynamic graph support through lazy recomputation of affected distance tables, (5) learning-based landmark placement optimized for observed query distributions [Velickovic and Blundell, 2021], (6) integration as a warm-start strategy for CH vertex ordering, and (7) extension to multi-criteria shortest paths with Pareto-optimal pruning.

Reproducibility. All code, data, and benchmark scripts are available in the accompanying repository with fixed random seeds, pinned dependencies, and a single-command reproduction pipeline via `make all`.

References

- Ralph Abboud, Radoslav Dimitrov, and Ismail Ilkan Ceylan. Shortest path networks for graph property prediction. In *First Learning on Graphs Conference (LoG)*, volume 198, 2022. URL <https://arxiv.org/abs/2206.01003>.
- Ittai Abraham, Daniel Delling, Andrew V. Goldberg, and Renato F. Werneck. Hierarchical hub labelings for shortest paths. In *European Symposium on Algorithms (ESA)*, pages 24–35. Springer, 2012. URL https://doi.org/10.1007/978-3-642-33090-2_4.
- Hannah Bast, Stefan Funke, Peter Sanders, and Dominik Schultes. Fast routing in road networks with transit nodes. In *Science*, volume 316, page 566, 2007. URL <https://doi.org/10.1126/science.1137521>.
- Richard Bellman. On a routing problem. *Quarterly of Applied Mathematics*, 16(1):87–90, 1958. URL <https://doi.org/10.1090/qam/102435>.

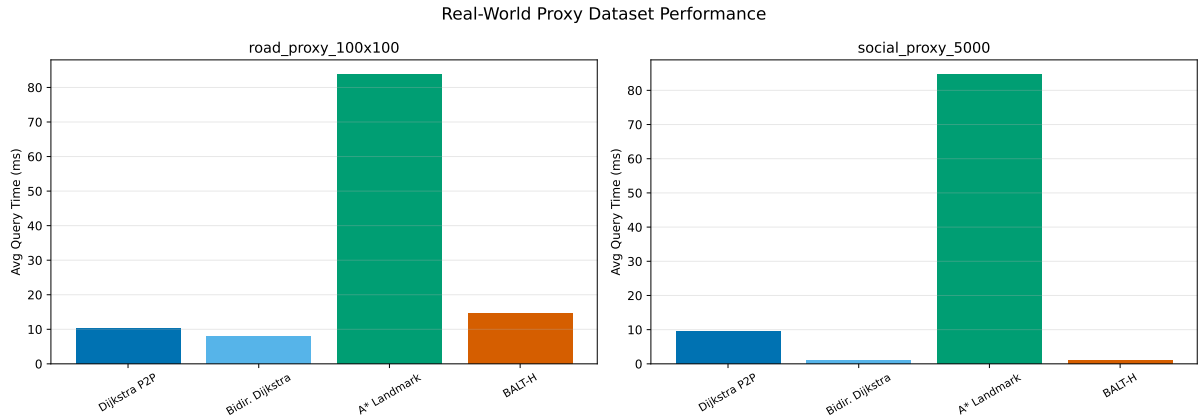


Figure 7: Performance on real-world proxy datasets with 100 random queries each. On the social network proxy, BALT-H achieves $10.5\times$ geometric mean speedup over Dijkstra P2P, with peak speedups exceeding $4,000\times$ on highly favorable query pairs where hub vertices provide near-exact upper bounds.

Daniel Delling, Andrew V. Goldberg, Thomas Pajor, and Renato F. Werneck. Customizable route planning. In *International Symposium on Experimental Algorithms (SEA)*, pages 376–387. Springer, 2011. URL https://doi.org/10.1007/978-3-642-20662-7_32.

Daniel Delling, Andrew V. Goldberg, Thomas Pajor, and Renato F. Werneck. Customizable route planning in road networks. *Transportation Science*, 51(2):566–591, 2017. URL <https://doi.org/10.1287/trsc.2014.0579>.

Robert B. Dial. Algorithm 360: Shortest-path forest with topological ordering. *Communications of the ACM*, 12(11):632–633, 1969. URL <https://doi.org/10.1145/363269.363610>.

E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1: 269–271, 1959. URL <https://doi.org/10.1007/BF01386390>.

Amr Elmasry. Breaking the bellman-ford shortest-path bound. *arXiv preprint*, 2019. URL <https://arxiv.org/abs/2410.23383>.

Robert W. Floyd. Algorithm 97: Shortest path. *Communications of the ACM*, 5(6):345, 1962. URL <https://doi.org/10.1145/367766.368168>.

Michael L. Fredman and Robert Endre Tarjan. Fibonacci heaps and their uses in improved network optimization algorithms. *Journal of the ACM*, 34(3):596–615, 1987. URL <https://doi.org/10.1145/28869.28874>.

Robert Geisberger, Peter Sanders, Dominik Schultes, and Daniel Delling. Contraction hierarchies: Faster and simpler hierarchical routing in road networks. In *International Workshop on Experimental and Efficient Algorithms (WEA)*, pages 319–333. Springer, 2008. URL https://doi.org/10.1007/978-3-540-68552-4_24.

Andrew V. Goldberg and Chris Harrelson. Computing the shortest path: A* search meets graph theory. In *Proceedings of the 16th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 156–165, 2005. URL <https://www.cs.princeton.edu/courses/archive/spr06/cos423/Handouts/GH05.pdf>.

P. E. Hart, N. J. Nilsson, and B. Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. URL <https://doi.org/10.1109/TSSC.1968.300136>.

- Donald B. Johnson. Efficient algorithms for shortest paths in sparse networks. *Journal of the ACM*, 24(1):1–13, 1977. URL <https://doi.org/10.1145/321992.321993>.
- Lester R. Ford Jr. Network flow theory. *RAND Corporation Paper*, (P-923), 1956. URL <https://www.rand.org/pubs/papers/P923.html>.
- Pan Li, Yanbang Wang, Hongwei Wang, and Jure Leskovec. Distance encoding: Design provably more powerful neural networks for graph representation learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020. URL <https://arxiv.org/abs/2009.00142>.
- Hector Ortega-Arranz, Y. Torres, A. Gonzalez-Escribano, and Diego R. Llanos. Comprehensive evaluation of a new gpu-based approach to the shortest path problem. *International Journal of Parallel Programming*, 43:918–938, 2015. URL <https://doi.org/10.1007/s10766-015-0351-z>.
- Ira Pohl. Bi-directional search. *Machine Intelligence*, 6:127–140, 1971.
- Petar Velickovic and Charles Blundell. Neural algorithmic reasoning. *Patterns*, 2(7), 2021. URL <https://doi.org/10.1016/j.patter.2021.100273>.
- Kai Wang, Don Fussell, and Calvin Lin. A fast work-efficient sssp algorithm for gpus. *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, pages 133–146, 2021. URL <https://www.cs.utexas.edu/~lin/papers/ppopp21.pdf>.
- Stephen Warshall. A theorem on boolean matrices. *Journal of the ACM*, 9(1):11–12, 1962. URL <https://doi.org/10.1145/321105.321107>.

Algorithm 1 BALT-H Query

Require: Graph G , source s , target t , preprocessing data $(\mathcal{L}, \mathcal{H})$

Ensure: Shortest path distance $d(s, t)$

```
1:  $\mu \leftarrow \min_{H \in \mathcal{H}} (d(H, s) + d(H, t))$  ▷ Hub upper bound
2:  $\text{active} \leftarrow \text{SELECTACTIVELANDMARKS}(s, t, k = 2)$  ▷ Optimization 1
3: Initialize forward heap  $Q_f \leftarrow \{(0, s)\}$ , distances  $g_f[s] \leftarrow 0$ 
4: Initialize backward heap  $Q_b \leftarrow \{(0, t)\}$ , distances  $g_b[t] \leftarrow 0$ 
5:  $S_f \leftarrow \emptyset, S_b \leftarrow \emptyset$  ▷ Settled sets
6: while  $Q_f \neq \emptyset$  or  $Q_b \neq \emptyset$  do
7:    $g_f^* \leftarrow \min(Q_f), g_b^* \leftarrow \min(Q_b)$  ▷ Minimum g-values
8:   if  $g_f^* + g_b^* \geq \mu$  then
9:     break ▷ Optimal termination
10:  end if
11:  if  $g_f^* \leq g_b^*$  then
12:    ▷ Expand smaller frontier  $(g, u) \leftarrow \text{EXTRACTMIN}(Q_f)$  if  $u \in S_f$  then continue
13:  end if
14:   $S_f \leftarrow S_f \cup \{u\}$ 
15:  if  $u \in g_b$  then
16:     $\mu \leftarrow \min(\mu, g + g_b[u])$ 
17:  end if
18:   $\text{lb} \leftarrow \max_{L \in \text{active}} (|d(L, t) - d(L, u)|, |d(u, L) - d(t, L)|)$ 
19:  if  $g + \text{lb} \geq \mu$  then
20:    continue ▷ Landmark pruning
21:  end if
22:  for all  $(v, w) \in \text{neighbors}(u)$  with  $v \notin S_f$  do
23:    if  $g + w < g_f[v]$  then
24:       $g_f[v] \leftarrow g + w$ ; push  $(g + w, v)$  to  $Q_f$ 
25:    if  $v \in g_b$  then
26:       $\mu \leftarrow \min(\mu, g + w + g_b[v])$ 
27:    end if
28:  end if
29:  end for
30:  else
31:    Symmetric backward expansion from  $Q_b$ 
32:  end if
33: end while
34: return  $\mu$ 
```
